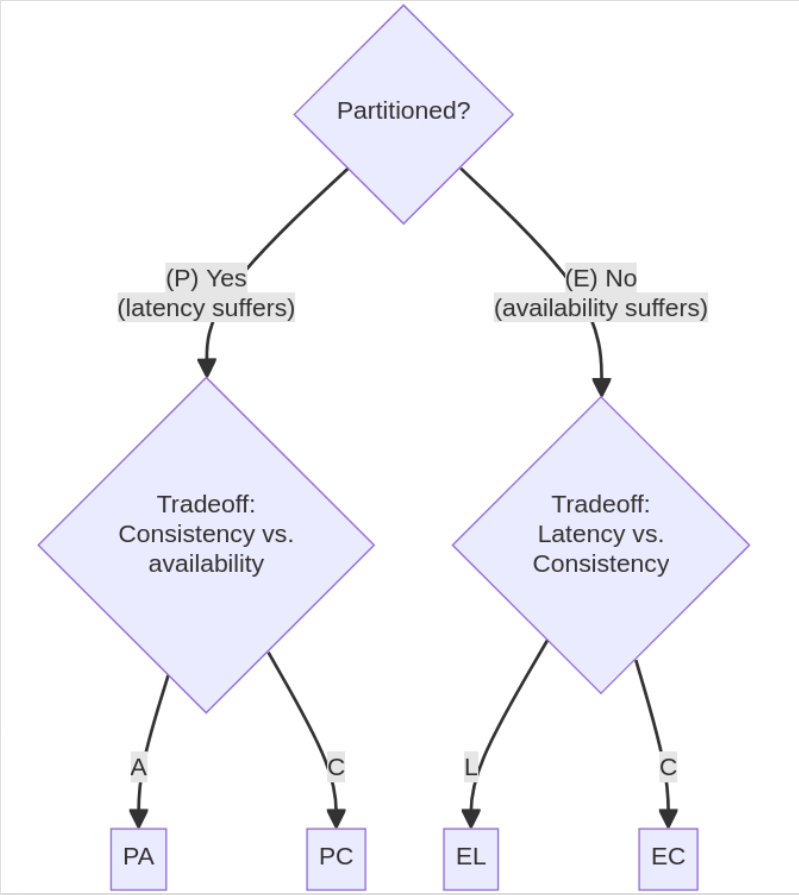
WIKIPEDIA
The Free Encyclopedia

PACELC design principle

In database theory, the **PACELC design principle** is an extension to the CAP theorem. It states that in case of network partitioning (P) in a distributed computer system, one has to choose between availability (A) and consistency (C) (as per the CAP theorem), but else (E), even when the system is running normally in the absence of partitions, one has to choose between latency (L) and loss of consistency (C).



The tradeoff between availability, consistency and latency, as described by the PACELC design principle.

Overview

The CAP theorem can be phrased as "PAC", the impossibility theorem that no distributed data store can be both consistent and available in executions that contains partitions. This can be proved by examining latency: if a system ensures consistency, then operation latencies grow with message delays, and hence operations cannot terminate eventually if the network is partitioned, i.e. the system cannot ensure availability.[1]

In the absence of partitions, both consistency and availability can be satisfied.[2] PACELC therefore goes further and examines how the system replicates data. Specifically, in the absence of partitions, an additional trade-off (ELC) exists between latency and consistency.[3] If the store is atomically consistent, then the sum of the read and write delay is at least the message delay. In practice, most systems rely on explicit acknowledgments rather than timed delays to ensure delivery, requiring a full network round trip and therefore message delay on both reads and writes to ensure consistency.[1] In low latency systems, in contrast, consistency is relaxed in order to reduce latency.[2]

There are four configurations or tradeoffs in the PACELC space:

- PA/EL - prioritize availability and latency over consistency
- PA/EC - when there is a partition, choose availability; else, choose consistency
- PC/EL - when there is a partition, choose consistency; else, choose latency
- PC/EC - choose consistency at all times

PC/EC and PA/EL provide natural cognitive models for an application developer. A PC/EC system provides a firm guarantee of atomic consistency, as in ACID, while PA/EL provides high availability and low latency with a more complex consistency model. In contrast, PA/EC and PC/EL systems only make conditional guarantees of consistency. The developer still has to write code to handle the cases where the guarantee is not upheld. PA/EC systems are rare outside of the in-memory data grid industry, where systems are localized to geographic regions and the latency vs. consistency tradeoff is not significant.[4] PC/EL is even more tricky to understand. PC does not indicate that the system is fully consistent; rather it indicates that the system does not reduce consistency beyond the baseline consistency level when a network partition occurs—instead, it reduces availability.[3]

Some experts like Marc Brooker argue that the CAP theorem is particularly relevant in intermittently connected environments, such as those related to the Internet of Things (IoT) and mobile applications. In these contexts, devices may become partitioned due to challenging physical conditions, such as power outages or when entering confined spaces like elevators. For distributed systems, such as cloud applications, it is more appropriate to use the PACELC theorem, which is more comprehensive and considers trade-offs such as latency and consistency even in the absence of network partitions.[5]

History

The PACELC theorem was first described by Daniel Abadi from Yale University in 2010 in a blog post,[2] which he later clarified in a paper in 2012.[3] The purpose of PACELC is to address his thesis that "Ignoring the consistency/latency trade-off of replicated systems is a major oversight [in CAP], as it is present at all times during system operation, whereas CAP is only relevant in the arguably rare case of a network partition." The PACELC theorem was proved formally in 2018 in a SIGACT News article.[1]

Database PACELC ratings

[3] Original database PACELC ratings are from.[6] Subsequent updates contributed by wikipedia community.

- The default versions of Amazon's early (internal) Dynamo (https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf), Cassandra, Riak, and Cosmos DB are PA/EL systems: if a partition occurs, they give up consistency for availability, and under normal operation they give up consistency for lower latency.
- Fully ACID systems such as VoltDB/H-Store, Megastore, MySQL Cluster, and PostgreSQL are PC/EC: they refuse to give up consistency, and will pay the availability and latency costs to achieve it. Bigtable and related systems such as HBase are also PC/EC.
- Amazon DynamoDB (launched January 2012) is quite different from the early (Amazon internal) Dynamo (https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf) which was considered for the PACELC paper.[6] DynamoDB follows a strong leader model, where every write is strictly serialized (and conditional writes carry no penalty) and supports read-after-write consistency. This guarantee does not apply to "Global Tables[7]" across regions. The DynamoDB

SDKs use eventually consistent reads by default (improved availability and throughput), but when a consistent read is requested the service will return either a current view to the item or an error.

- Couchbase provides a range of consistency and availability options during a partition, and equally a range of latency and consistency options with no partition. Unlike most other databases, Couchbase doesn't have a single API set nor does it scale/replicate all data services homogeneously. For writes, Couchbase favors Consistency over Availability making it formally CP, but on read there is more user-controlled variability depending on index replication, desired consistency level and type of access (single document lookup vs range scan vs full-text search, etc.). On top of that, there is then further variability depending on cross-datacenter-replication (XDCR) which takes multiple CP clusters and connects them with asynchronous replication and Couchbase Lite which is an embedded database and creates a fully multi-master (with revision tracking) distributed topology.
- Cosmos DB supports five tunable consistency levels that allow for tradeoffs between C/A during P, and L/C during E. Cosmos DB never violates the specified consistency level, so it's formally CP.
- MongoDB can be classified as a PA/EC system. In the baseline case, the system guarantees reads and writes to be consistent.
- PNUTS is a PC/EL system.
- Hazelcast IMDG and indeed most in-memory data grids are an implementation of a PA/EC system; Hazelcast can be configured to be EL rather than EC.[8] Concurrency primitives (Lock, AtomicReference, CountDownLatch, etc.) can be either PC/EC or PA/EC.[9]
- FaunaDB implements Calvin, a transaction protocol created by Dr. Daniel Abadi, the author[3] of the PACELC theorem, and offers users adjustable controls for LC tradeoff. It is PC/EC for strictly serializable transactions, and EL for serializable reads.

DDBS	P+A	P+C	E+L	E+C
Aerospike[10]	✓	paid only	optional	✓
Bigtable/HBase		✓		✓
Cassandra	✓		✓[a]	✓[a]
Cosmos DB		✓	✓[b]	
Couchbase	✓		✓	✓
Dynamo	✓		✓[a]	
DynamoDB		✓	✓	✓
FaunaDB[12]		✓	✓	✓
Hazelcast IMDG[8][9]	✓	✓	✓	✓
Megastore		✓		✓
MongoDB	✓			✓
MySQL Cluster		✓		✓
PNUTS		✓	✓	
PostgreSQL	✓	✓	✓	✓
Riak	✓		✓[a]	
SpiceDB[13]		✓	✓	✓
VoltDB/H-Store		✓		✓

See also

- CAP theorem
- Consistency model
- Fallacies of distributed computing
- Lambda architecture (solution)
- Paxos (computer science)
- Project management triangle
- Raft (algorithm)
- Trilemma

Notes

a. Dynamo, Cassandra, and Riak have user-adjustable settings to control the LC tradeoff.[6]
b. Cosmos DB has five selectable consistency levels to control the LC tradeoff.[11]

References

1. Golab, Wojciech (2018). "Proving PACELC" (https://dl.acm.org/doi/10.1145/3197406.3197420). *ACM SIGACT News*. **49** (1): 73–81. doi:10.1145/3197406.3197420 (https://doi.org/10.1145%2F3197406.3197420). S2CID 3989621 (https://api.semanticscholar.org/CorpusID:3989621).
2. Abadi, Daniel J. (2010-04-23). "DBMS Musings: Problems with CAP, and Yahoo's little known NoSQL system" (https://dbmsmusings.blogspot.com/2010/04/problems-with-cap-and-yahoos-little.html). Retrieved 2016-09-11.
3. Abadi, Daniel J. "Consistency Tradeoffs in Modern Distributed Database System Design" (http://www.cs.umd.edu/~abadi/papers/abadi-pacelc.pdf) (PDF). Yale University.
4. Abadi, Daniel (15 July 2019). "The dangers of conditional consistency guarantees" (https://dbmsmusings.blogspot.com/2019/07/the-dangers-of-conditional-consistency.html). *DBMS Musings*. Retrieved 29 August 2024.
5. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media. ISBN 978-1449373320.
6. Abadi, Daniel J.; Murdopo, Arinto (2012-04-17). "Consistency Tradeoffs in Modern Distributed Database System Design" (https://www.slideshare.net/arinto/consistency-tradeoffinmoderndb). Retrieved 2022-07-18.
7. "Global tables - multi-Region replication for DynamoDB" (https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GlobalTables.html). *AWS Documentation*. Retrieved 4 January 2023.
8. Abadi, Daniel (2017-10-08). "DBMS Musings: Hazelcast and the Mythical PA/EC System" (https://dbmsmusings.blogspot.com/2017/10/hazelcast-and-mythical-paec-system.html). *DBMS Musings*. Retrieved 2017-10-20.
9. "Hazelcast IMDG Reference Manual" (https://docs.hazelcast.org/docs/4.0.2/manual/html-single/index.html#cp-subsystem). *docs.hazelcast.org*. Retrieved 2020-09-17.
10. Porter, Kevin (29 March 2023). "Where does aerospike fall in PACELC?" (https://discuss.aerospike.com/t/where-does-aerospike-fall-in-pacelc/10111/2). *Aerospike Community Forum*. Retrieved 30 March 2023.

11. "Consistency Levels in Azure Cosmos DB" (https://learn.microsoft.com/en-us/azure/cosmos-db/consistency-levels). Retrieved 2021-06-21.
12. Abadi, Daniel (2018-09-21). "DBMS Musings: NewSQL database systems are failing to guarantee consistency, and I blame Spanner" (https://dbmsmusings.blogspot.com/2018/09/newsql-database-systems-are-failing-to.html). *DBMS Musings*. Retrieved 2019-02-23.
13. Zelinskie, Jimmy (2024-04-23). "SpiceDB Concepts: Consistency" (https://authzed.com/docs/spicedb/concepts/consistency). *SpiceDB documentation*. Retrieved 2024-05-02.

External links

- "Consistency Tradeoffs in Modern Distributed Database System Design", by Daniel J. Abadi, Yale University (https://www.cs.umd.edu/~abadi/papers/abadi-pacelc.pdf) Original paper that formalized PACELC
- "Problems with CAP, and Yahoo's little known NoSQL system", by Daniel J. Abadi, Yale University (https://dbmsmusings.blogspot.com/2010/04/problems-with-cap-and-yahoos-little.html). Original blog post that first described PACELC
- "Proving PACELC", by Wojciech Golab, University of Waterloo (https://uwaterloo.ca/distributed-algorithms-systems-lab/sites/default/files/uploads/files/proving_pacelc.pdf) Formal proof of the PACELC theorem